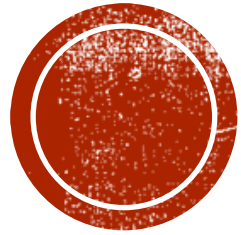


INGENIERÍA DE SOFTWARE I RE - DICTADO

- 2026 -

Modelos de Procesos
Modelos Tradicionales
Metodologías ágiles
Calidad





MODELO DE PROCESOS - TRADICIONALES

MODELO DE PROCESO

»Proceso

- Actividades que involucran, restricciones y recursos que producen una determinada salida
- Características
 - Establece todas las actividades
 - Utiliza recursos, está sujeto a restricciones y genera productos intermedios y finales
 - Puede estar compuesto por subprocesos
 - Cada actividad tiene entradas y salidas definidas
 - Las actividades se organizan en una secuencia
 - Existen principios que orientan sobre las metas de cada actividad
 - Las restricciones pueden aplicarse a una actividad, recurso o producto.



MODELO DE PROCESO

»Ciclo de vida

- Proceso que implica la construcción de un producto

»Ciclo de vida del Software

- Describe la vida del producto de software desde su concepción hasta su implementación, entrega utilización y mantenimiento

»Modelos de proceso de software

- Es una representación abstracta de un proceso del software.
 - Modelo de proceso
 - Paradigma de software
 - Ciclo de vida del software

Términos Equivalentes



MODELO DE PROCESO

» Modelos prescriptivos

- Prescriben un conjunto de elementos del proceso: actividades del marco de trabajo, acciones de la ingeniería del software, tareas, aseguramiento de la calidad y mecanismos de control.
- Cada modelo de proceso prescribe también un “flujo de trabajo”, es decir de que forma los elementos del proceso se interrelacionan entre sí.

» Modelos descriptivos

- Descripción en la forma en que se realizan en la realidad

» Ambos modelos deberían ser iguales



MODELO DE PROCESO

- »Modelo en Cascada
- »Modelo en V
- »Modelo de Prototipos
- »Desarrollo por fases
 - Incremental
 - Iterativo
- »El modelo espiral



MODELO DE PROCESO

»Modelo en Cascada

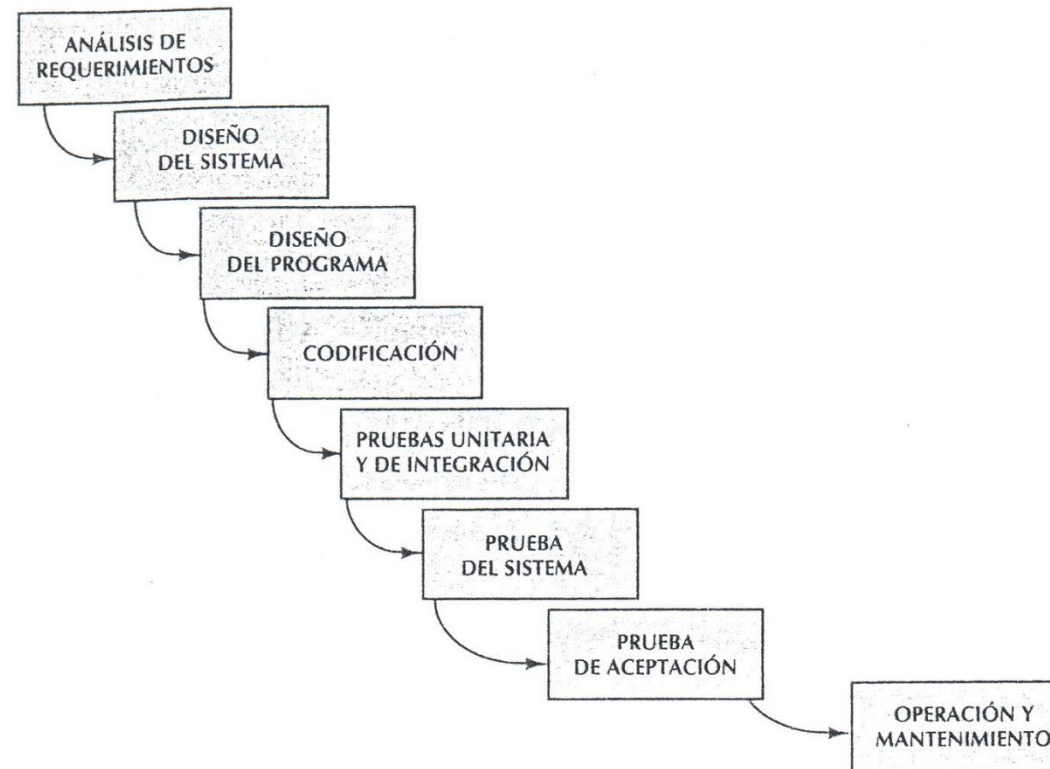
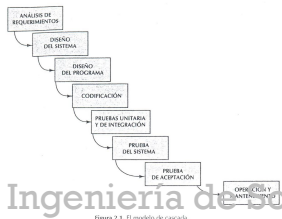


Figura 2.1. El modelo de cascada.

MODELO DE PROCESO

»Modelo en Cascada

- Las etapas se representan cayendo en cascada
- Cada etapa de desarrollo se debe completar antes que comience la siguiente
- Útil para diagramar lo que se necesita hacer
- Su simplicidad hace que sea fácil explicarlo a los clientes



MODELO DE PROCESO

»Modelo en Cascada

■ Dificultades:

- No existen resultados concretos hasta que todo este terminado.
- Las fallas más triviales se encuentran al comienzo del período de prueba y las más graves al final.
- La eliminación de fallas suele ser extremadamente difícil durante las últimas etapas de prueba del sistema.
- Deriva del mundo del hardware y presenta una visión de manufactura sobre el desarrollo de software.
- El software no se desarrolla de la misma manera
- La necesidad de prueba con la computadora aumenta exponencialmente durante las etapas finales de prueba.
- "CONGELAR" una fase es poco realista.
- Existen errores, cambios de parecer, cambios en el ambiente.



MODELO DE PROCESO

»Modelo de la realidad (sin control entre las etapas)

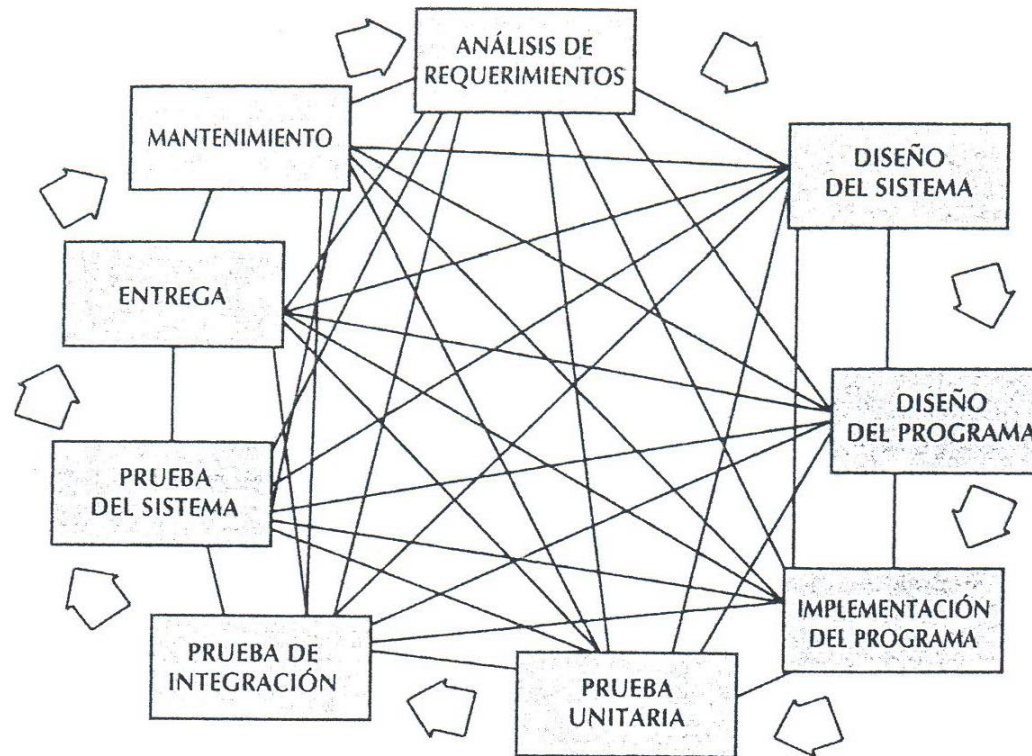


Figura 2.2. El proceso de desarrollo del software en la realidad.
Fuente: Freyge

MODELO DE PROCESO

»Modelo en Cascada

■ Con prototipo

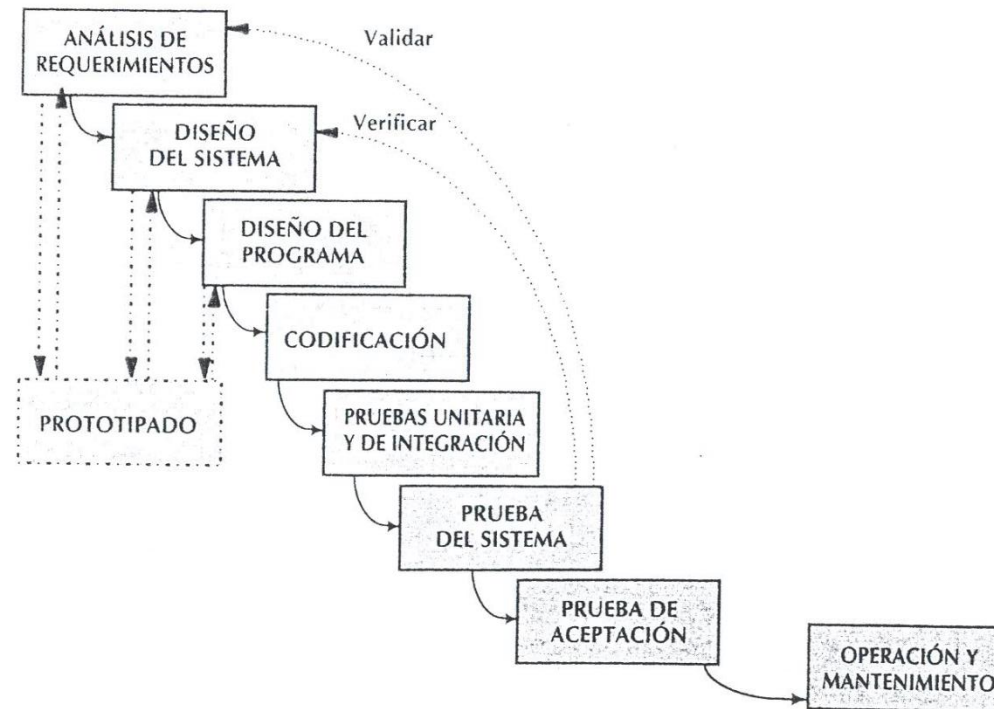
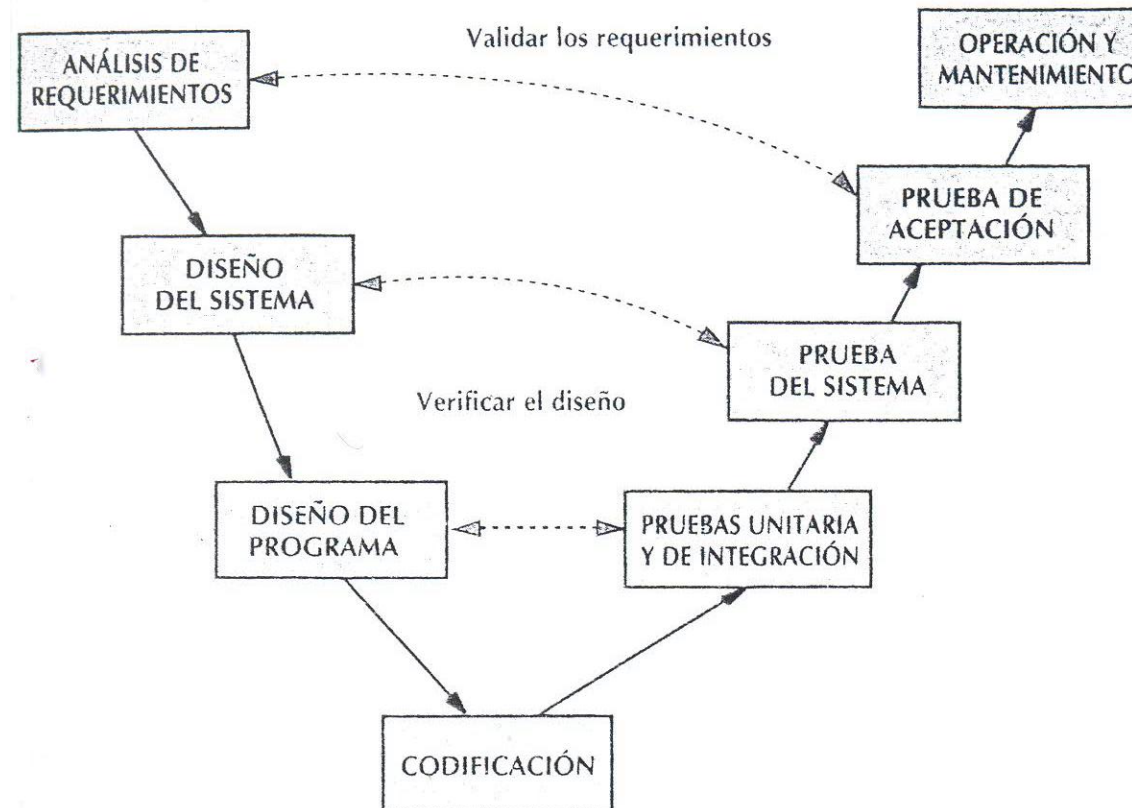


Figura 2.3. El modelo de cascada con prototipado.

MODELO DE PROCESO

»Modelo en V



MODELO DE PROCESO

»Modelo en V

- Demuestra como se relacionan las actividades de prueba con las de análisis y diseño.
- Sugiere que la prueba unitaria y de integración también sea utilizada para verificar el diseño del programa
- La vinculación entre los lados derecho e izquierdo implica que, si se encuentran problemas durante la verificación y validación, entonces el lado izquierdo de la V puede ser ejecutado nuevamente para solucionar el problema.

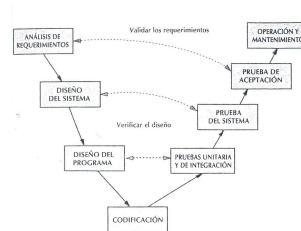


Figura 2.4. El modelo V.



MODELO DE PROCESO

»Modelo de Prototipos

- Un prototipo es un producto parcialmente desarrollado que permite que clientes y desarrolladores examinen algunos aspectos del sistema propuesto, y decidan si éste es adecuado o correcto para el producto terminado.
- Esta es una alternativa de especificación para tratar mejor la incertidumbre, la ambigüedad y la volubilidad de los proyectos reales.

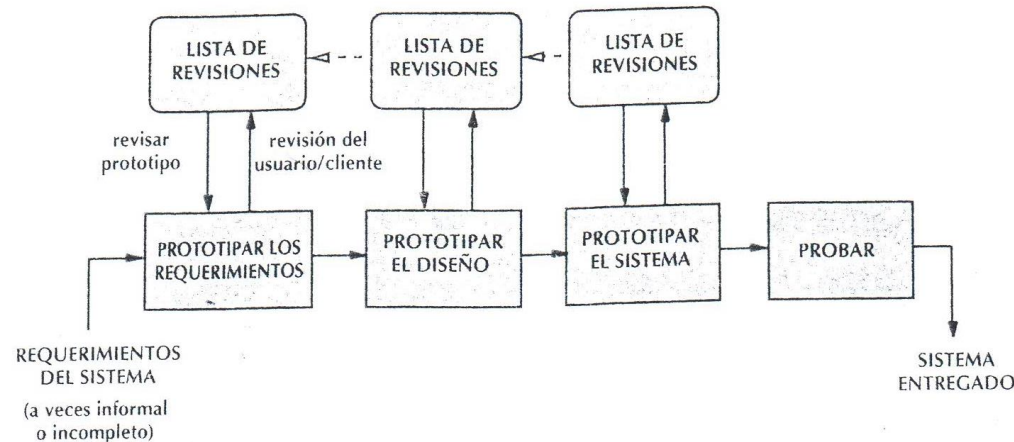


Figura 2.5. El modelo prototipado.

MODELO DE PROCESO

»Modelo de Prototipos

■ Tipos

■ Evolutivos

- El objetivo es obtener el sistema a entregar.
- Permite que todo el sistema o alguna de sus partes se construyan rápidamente para comprender o aclarar aspectos y asegurar que el desarrollador, el usuario y el cliente tengan una comprensión unificada tanto de lo que se necesita como de lo que se propone como solución

■ Descartables

- No tiene funcionalidad
- Se utilizan herramientas de modelado

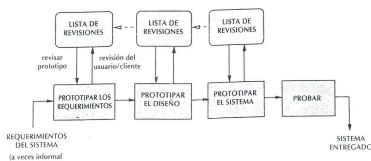


Figura 2.3. El modelo prototipado.

MODELO DE PROCESO

»Modelo de Prototipos

- Proyectos candidatos
 - Usuarios que no examinarán los modelos abstractos
 - Usuarios que no determinarán sus requerimientos inicialmente
 - Sistemas con énfasis en los formatos de E/S más que en los detalles algorítmicos
 - Sistemas en los que haya que explorar aspectos técnicos
 - Si el usuario tiene dificultad al tratar con los modelos gráficos para modelar los requerimientos y el comportamiento
 - Si se enfatiza el aspecto de la interfaz humana



Figura 2.5. El modelo prototipado.

MODELO DE PROCESO

»Modelo de Prototipos

- Para asegurar el éxito
 - Debe ser un sistema con el que se pueda experimentar
 - Debe ser comparativamente barato ($< 10\%$)
 - Debe desarrollarse rápidamente
 - Énfasis en la interfaz de usuario
 - Equipo de desarrollo reducido
 - Herramientas y lenguajes adecuados

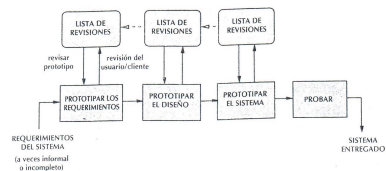


Figura 2.5. El modelo prototipo.

MODELO DE PROCESO

»Desarrollo por fases

- Se desarrolla el sistema de tal manera que puede ser entregado en piezas. Esto implica que existen dos sistemas funcionando en paralelo: el sistema operacional y el sistema en desarrollo.

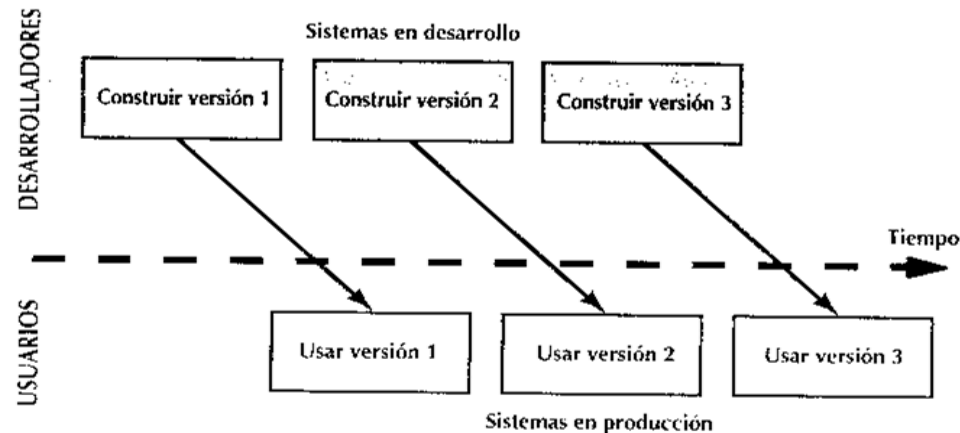


Figura 2.8. El modelo de desarrollo escalonado.

MODELO DE PROCESO

»Desarrollo por fases

- Incremental
 - El sistema es particionado en subsistemas de acuerdo con su funcionalidad. Cada entrega agrega un subsistema.
- Iterativo
 - Entrega un sistema completo desde el principio y luego aumenta la funcionalidad de cada subsistema con las nuevas versiones

Sistema completo con funcionalidad básica

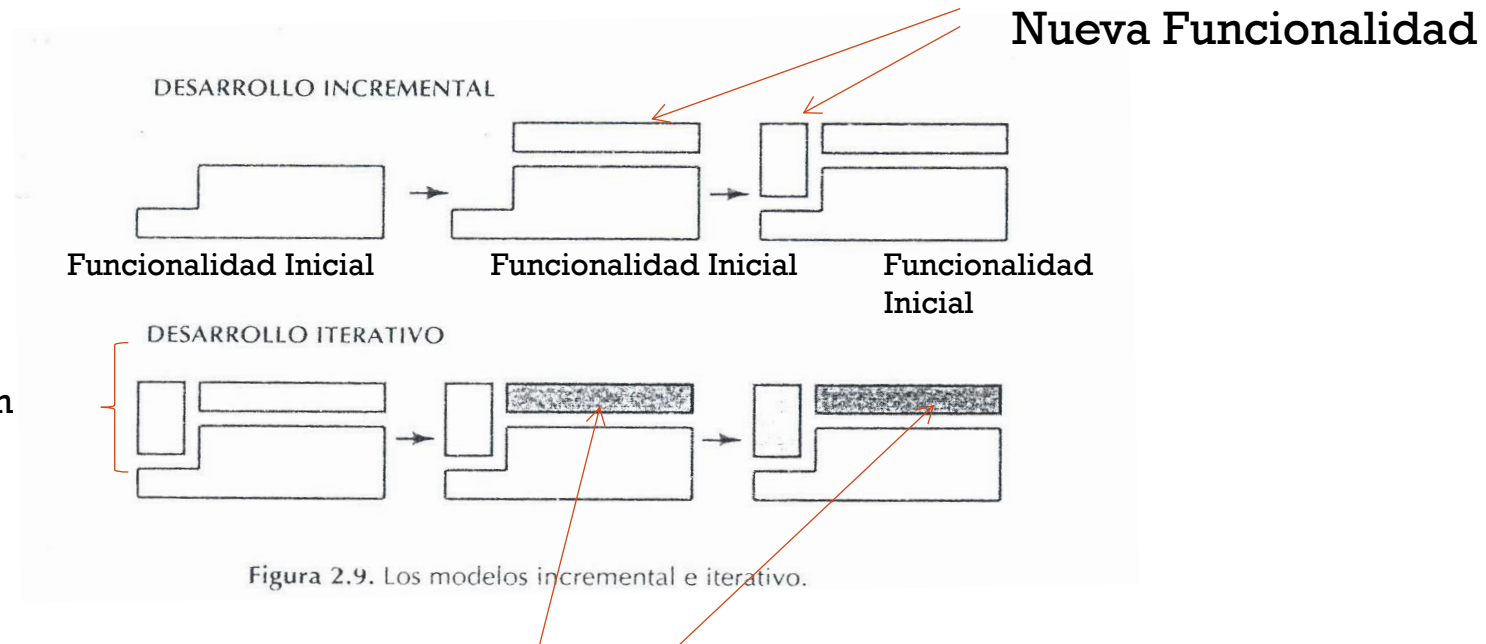


Figura 2.9. Los modelos incremental e iterativo.

MODELO DE PROCESO

»El modelo espiral (Boehm)

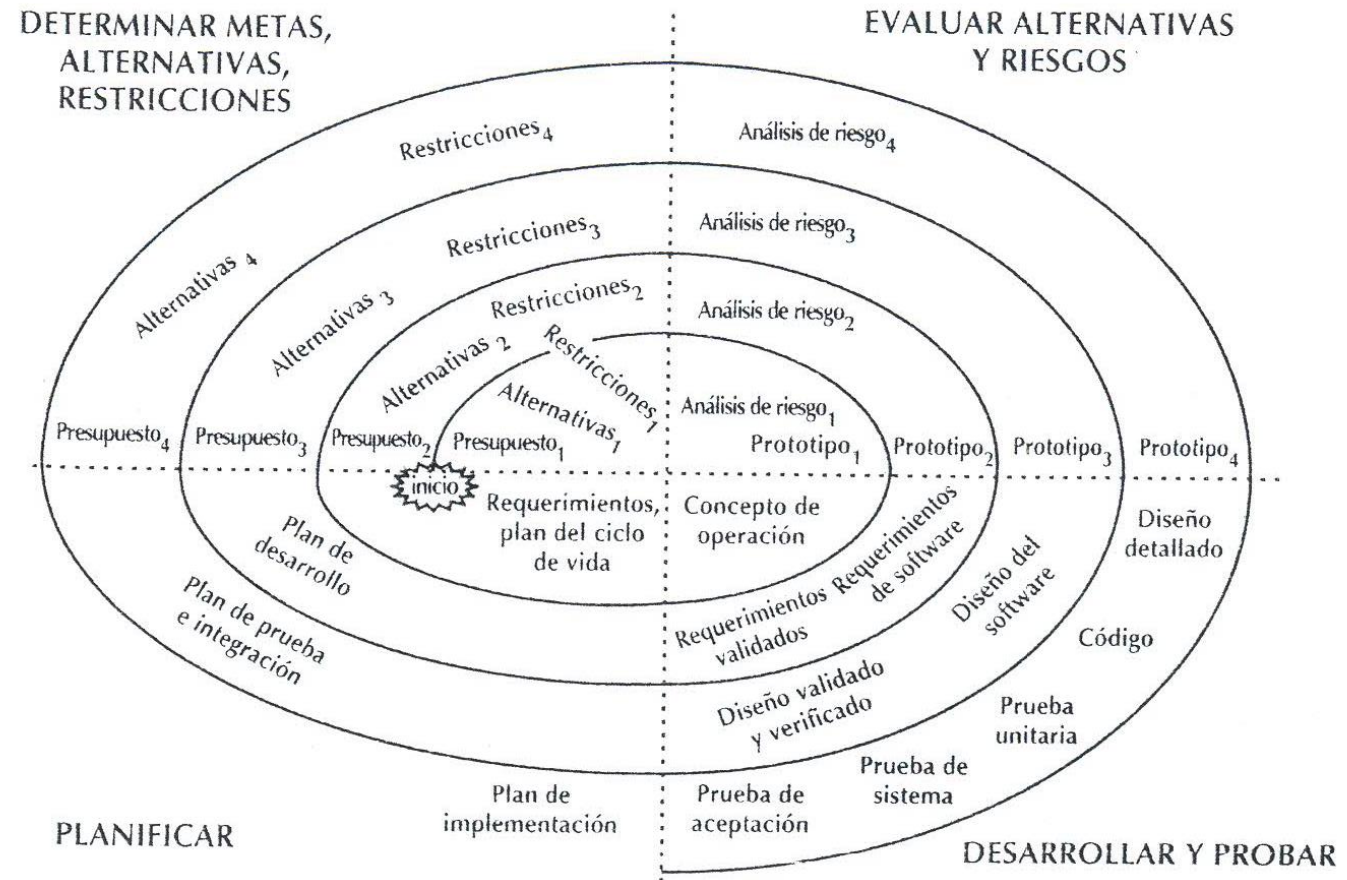


Figura 2.10. El modelo en espiral.

MODELO DE PROCESO

»El modelo espiral (Boehm)

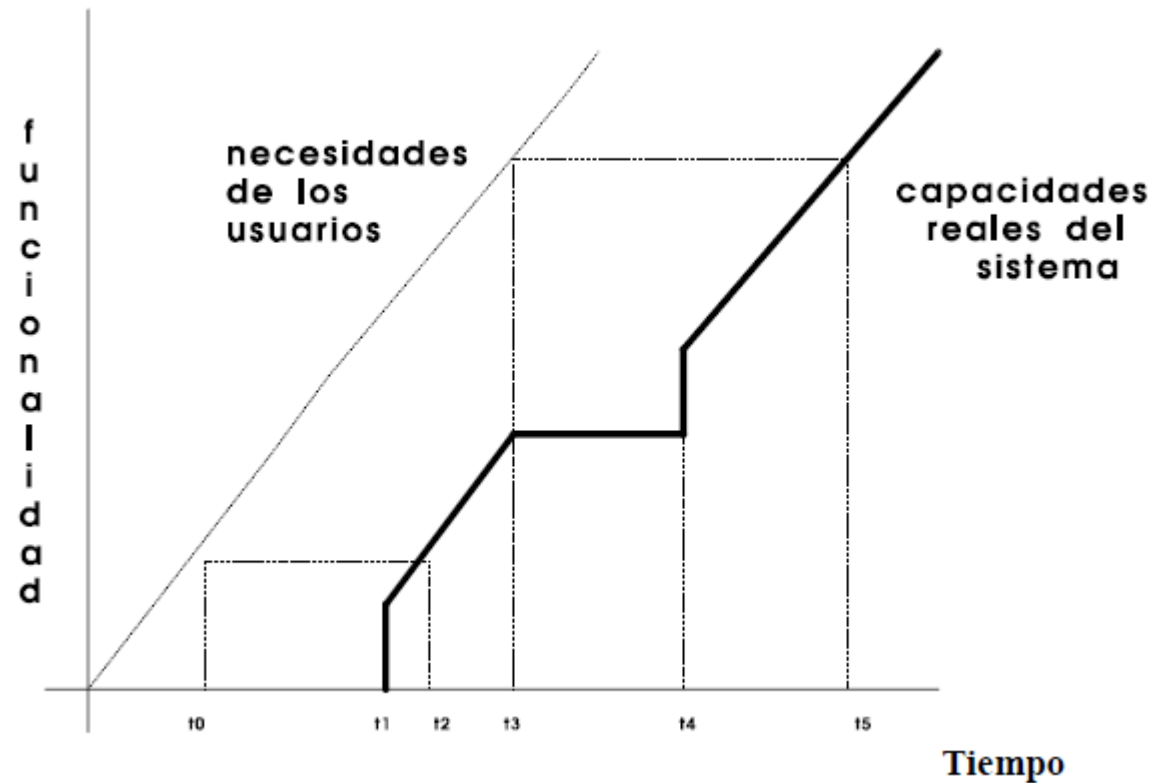
- Combina las actividades de desarrollo con la gestión del riesgo
- Trata de mejorar los ciclos de vida clásicos y prototipos.
- Incorpora objetivos de calidad y gestión de riesgos
- Elimina errores y alternativas no atractivas al comienzo
- Permite iteraciones, vuelta atrás y finalizaciones rápidas
- Cada ciclo empieza identificando:
 - Los objetivos de la porción correspondiente
 - Las alternativas
 - Restricciones
- Cada ciclo se completa con una revisión que incluye todo el ciclo anterior y el plan para el siguiente



MODELO DE PROCESO

»Análisis Comparativo

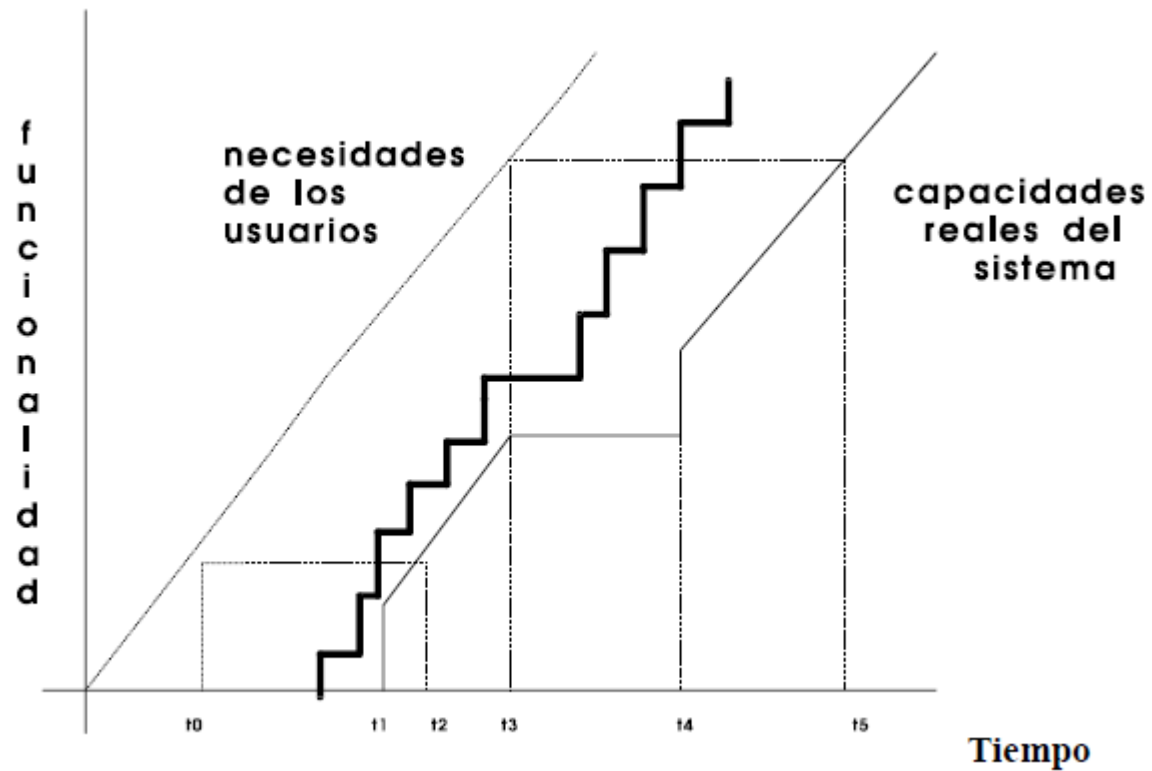
Clásico



MODELO DE PROCESO

»Análisis Comparativo

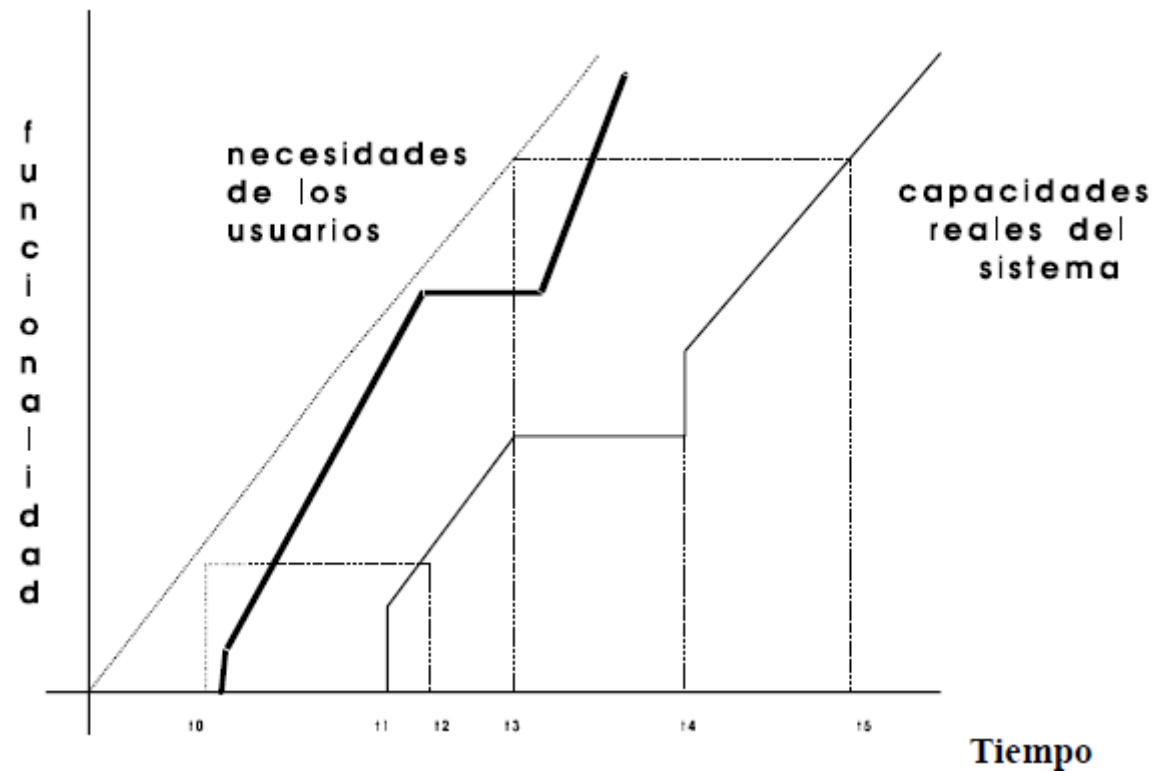
Incremental

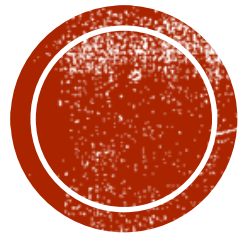


MODELO DE PROCESO

»Análisis Comparativo

Prototipo Evolutivo





MODELO DE PROCESOS - METODOLOGÍAS AGILES

METODOLOGÍAS AGILES

»La ingeniería de software ágil combina una filosofía y un conjunto de directrices de desarrollo.

- La filosofía busca
 - La satisfacción del cliente y la entrega temprana del software incremental
 - Equipos de proyectos pequeños y con alta motivación
 - Métodos informales un mínimo de productos de trabajo de la ingeniería de software
 - Una simplicidad en general
- Las directrices resaltan
 - La entrega sobre el análisis y el diseño, aunque estas actividades no se descartan
 - La comunicación activa y continua entre los desarrolladores y el cliente



METODOLOGÍAS AGILES

- » El desarrollo ágil e software son métodos de ingeniería del software basados en el desarrollo iterativo e incremental, donde los requerimientos y soluciones evolucionan mediante la colaboración de grupo auto organizados y multidisciplinares.
- » Existen muchos métodos de desarrollo ágil
 - la mayoría minimiza riesgos desarrollando software en cortos lapsos de tiempo.
 - El software desarrollado en una unidad de tiempo es llamado una iteración, la cual debe durar de una a cuatro semanas. Cada iteración del ciclo de vida incluye:
 - Planificación, análisis de requerimientos, diseño, codificación, revisión y documentación.
 - Una iteración no debe agregar demasiada funcionalidad para justificar el lanzamiento del producto al mercado, pero la meta es tener una «demo» (sin errores) al final de cada iteración.
 - Al final de cada iteración el equipo vuelve a evaluar las prioridades del proyecto.



METODOLOGÍAS AGILES

»La definición moderna de desarrollo ágil de software evolucionó a mediados de los años 1990 y en el año 2001, miembros prominentes de la comunidad se reunieron en Snowbird, Utah, y se creó la “*alianza ágil*”, una organización sin fines de lucro que promueve el desarrollo ágil de aplicaciones y firmaron lo que se denominó el “*Manifiesto para el desarrollo ágil de software*”



METODOLOGÍAS AGILES

»Manifiesto

- Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:
 - **Individuos e interacciones** sobre procesos y herramientas
 - **Software funcionando** sobre documentación extensiva
 - **Colaboración con el cliente** sobre negociación contractual
 - **Respuesta ante el cambio** sobre seguir un plan
- Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.



METODOLOGÍAS AGILES

»Manifiesto

1. Nuestra principal prioridad es satisfacer al cliente a través de la entrega temprana y continua de software de valor.
2. Son bienvenidos los requisitos cambiantes, incluso si llegan tarde al desarrollo. Los procesos ágiles se doblan al cambio como ventaja competitiva para el cliente.
3. Entregar con frecuencia software que funcione, en periodos de un par de semanas hasta un par de meses, con preferencia en los periodos breves.
4. Las personas del negocio y los desarrolladores deben trabajar juntos de forma cotidiana a través del proyecto.
5. Construcción de proyectos en torno a individuos motivados, dándoles la oportunidad y el respaldo que necesitan y procurándoles confianza para que realicen la tarea.
6. La forma más eficiente y efectiva de comunicar información de ida y vuelta dentro de un equipo de desarrollo es mediante la conversación cara a cara.



METODOLOGÍAS AGILES

»Manifiesto

7. El software que funciona es la principal medida del progreso.
8. Los procesos ágiles promueven el desarrollo sostenido. Los patrocinadores, desarrolladores y usuarios deben mantener un ritmo constante de forma indefinida.
9. La atención continua a la excelencia técnica enaltece la agilidad.
10. La simplicidad como arte de maximizar la cantidad de trabajo que no se hace, es esencial.
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos que se auto-organizan.
12. En intervalos regulares, el equipo reflexiona sobre la forma de ser más efectivo y ajusta su conducta en consecuencia.



METODOLOGÍAS ÁGILES

Metodologías Ágiles	Metodologías Tradicionales
Basadas en heurísticas provenientes de prácticas de producción de código	Basadas en normas provenientes de estándares seguidos por el entorno de desarrollo
Especialmente preparados para cambios durante el proyecto	Cierta resistencia a los cambios
Impuestas internamente (por el equipo)	Impuestas externamente
Proceso menos controlado, con pocos principios	Proceso mucho más controlado, con numerosas políticas/normas
No existe contrato tradicional o al menos es bastante flexible	Existe un contrato prefijado
El cliente es parte del equipo de desarrollo	El cliente interactúa con el equipo de desarrollo mediante reuniones
Grupos pequeños (<10 integrantes) y trabajando en el mismo sitio	Grupos grandes y posiblemente distribuidos
Pocos artefactos	Más artefactos
Pocos roles	Más roles
Menos énfasis en la arquitectura del software	La arquitectura del software es esencial y se expresa mediante modelos

Tabla 1. Diferencias entre metodologías ágiles y no ágiles



METODOLOGÍAS AGILES

» Metodologías mas relevantes

- XP – extreme programming
- Crystal
- ASD -Adaptative software development
- Scrum



EXTREME PROGRAMMING

- » Es una disciplina de desarrollo de software basado en los valores de la *sencillez*, la *comunicación*, la *retroalimentación*, la *valentía* y el *respeto*
- » Su acción consiste en llevar a todo el equipo reunido en la presencia de prácticas simples, con suficiente información para que el equipo para ver dónde están y para ajustar las prácticas a su situación particular.
- » En la programación extrema, cada colaborador del proyecto es una parte integral del "Equipo".
 - Las formas del equipo en torno a un representante de la empresa llama "el Cliente", que se sienta con el equipo y trabaja con ellos todos los días.



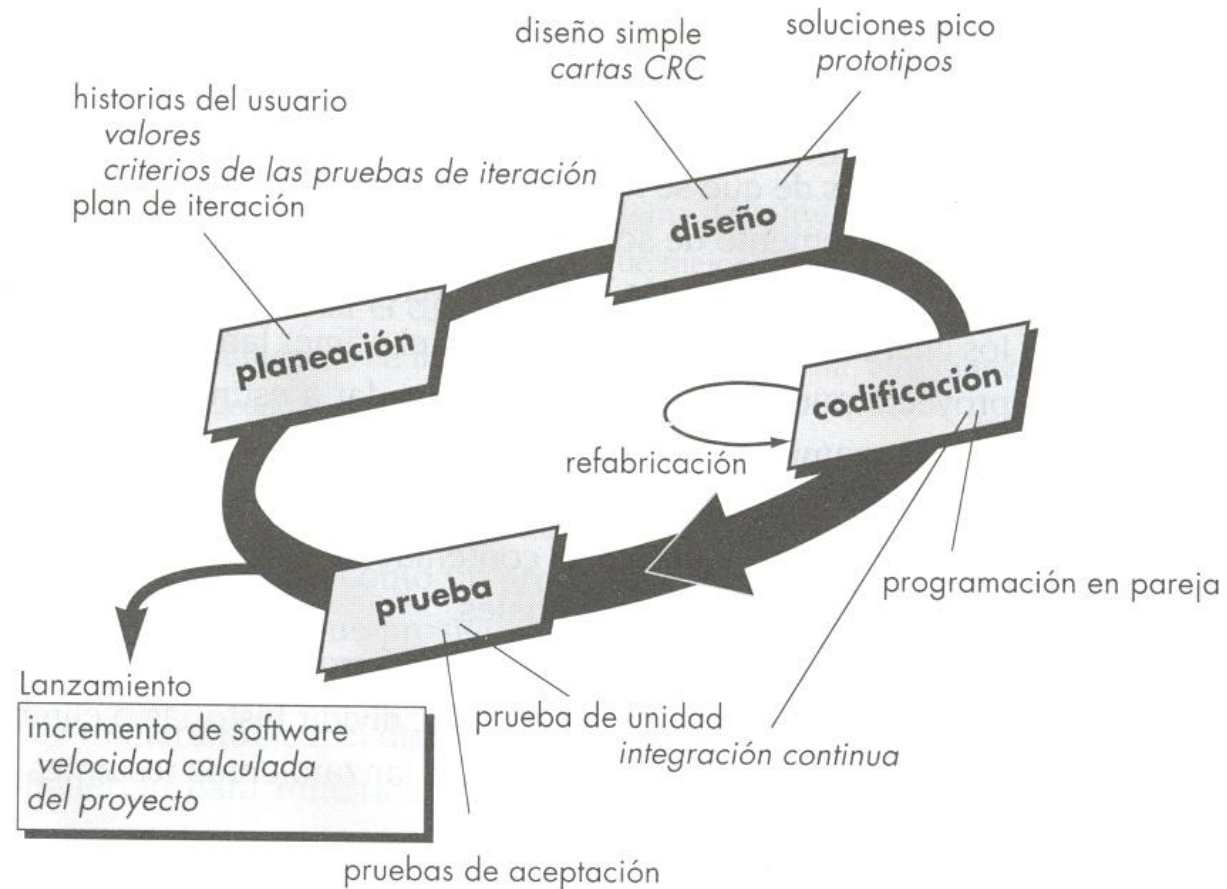
EXTREME PROGRAMMING

- Desarrollo iterativo e incremental: pequeñas mejoras, unas tras otras.
- Pruebas unitarias continuas, frecuentemente repetidas y automatizadas, incluyendo pruebas de regresión.
- Programación en parejas
- Frecuente integración del equipo de programación con el cliente o usuario.
- Corrección de todos los errores antes de añadir nueva funcionalidad.
- Refactorización del código
- Propiedad del código compartida
- Simplicidad en el código



EXTREME PROGRAMMING

»eXtreme Programming



SCRUM

- » Scrum es un proceso en el que se aplican, de manera regular, un conjunto de mejores prácticas para trabajar en equipo y obtener el mejor resultado posible de un proyecto.
- » Estas prácticas se apoyan unas a otras y su selección tiene origen en un estudio de la manera de trabajar de equipos altamente productivos.
- » En Scrum se realizan entregas parciales y regulares del resultado final del proyecto, priorizadas por el beneficio que aportan al receptor del proyecto



SCRUM

» Eliminar el desperdicio

- No generar artefactos, ni perder el tiempo haciendo cosas que no le suman valor al cliente.

» Construir la calidad con el producto

- La idea es inyectar la calidad directamente en el código desde el inicio.

» Crear conocimiento

- En la práctica no se puede tener el conocimiento antes de empezar el desarrollo.

» Diferir las decisiones

- Tomar las decisiones en el momento adecuado, esperar hasta ese momento, ya que uno tiene mas información a medida que va pasando el tiempo. Si se puede esperar, mejor.

» Entregar rápido

- Debe ser una de las ventajas competitivas más importantes.

» Respetar a las personas

- La gente trabaja mejor cuando se encuentra en un ambiente que la motive y se sienta respetada.

» Optimizar el todo

- Optimizar todo el proceso, ya que el proceso es una unidad, y para lograr tener éxito y avanzar, hay que tratarlo como tal.



SCRUM

»El Product Owner (Propietario)

- Conoce y marca las prioridades del proyecto o producto.



»El Scrum Master (Jefe)

- Es la persona que asegura el seguimiento de la metodología guiando las reuniones y ayudando al equipo ante cualquier problema que pueda aparecer. Su responsabilidad es entre otras, la de hacer de paraguas ante las presiones externas.



»El Scrum Team (Equipo)

- Son las personas responsables de implementar la funcionalidad o funcionalidades elegidas por el Product Owner.



»Los Usuarios o Clientes

- Son los beneficiarios finales del producto, y son quienes viendo los progresos, pueden aportar ideas, sugerencias o necesidades



SCRUM

»Product Backlog

- Es la lista maestra que contiene toda la funcionalidad deseada en el producto. La característica más importante es que la funcionalidad se encuentra ordenada por un orden de prioridad.

»Sprint Backlog

- Es la lista que contiene toda la funcionalidad que el equipo se comprometió a desarrollar durante un Sprint determinado.

»Burndown Chart

- Muestra un acumulativo del trabajo hecho, día-a-día.

»Entre otros...

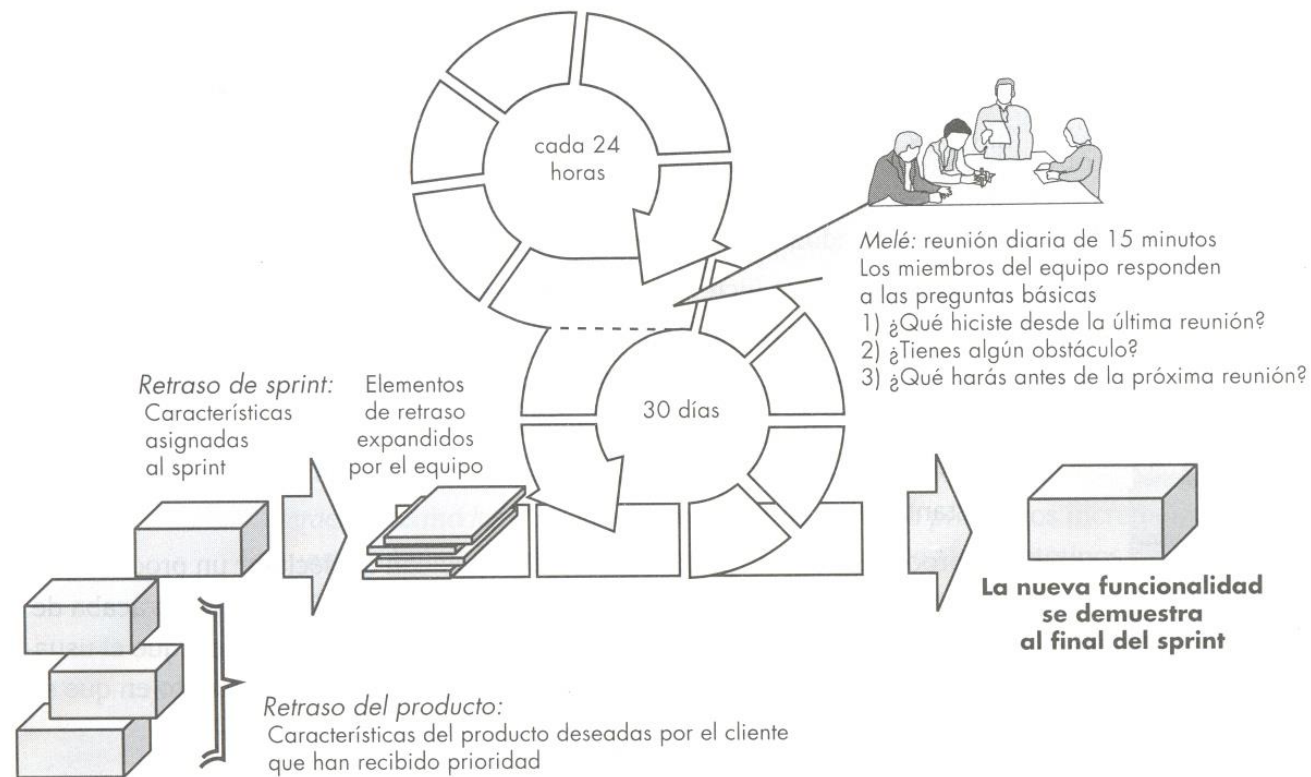
SCRUM

»Scrum es iterativo e incremental

- Se busca poder atacar todos los problemas que surgen durante el desarrollo del proyecto
- El nombre Scrum se debe a que durante los Sprints, lo que serían las fases de desarrollo, se solapan, de manera que no es un proceso de cascada por cada iteración, si no que tenemos todas éstas etapas juntas que se ejecutan una y otra vez, hasta que se crea suficiente.
- Este solapamiento de fases se puede asemejar a un scrum de rugby, en el cual todos los jugadores (o roles, en nuestro caso), trabajan juntos para lograr un objetivo.

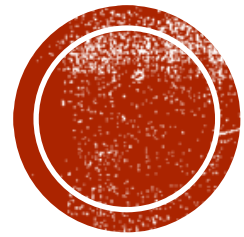


SCRUM - PROCESO



SCRUM

- » Scrum está pensado para ser aplicado en proyectos en donde el caos es una constante, aquellos proyectos en los que tenemos requerimientos dinámicos, y que tenemos que implementar tecnología de punta.
- » Esos proyectos difíciles, que con los enfoques tradicionales se hace imposible llegar a buen puerto.



CALIDAD DE SOFTWARE

¿QUE ES LA CALIDAD?

- » Calidad es un concepto manejado con bastante frecuencia en la actualidad, pero a su vez, su significado es percibido de distintas maneras.
- » Al hablar de bienes y/o servicios de calidad, la gente se refiere normalmente a bienes de lujo o excelentes con precios elevados.
- » Su significado sigue siendo ambiguo y muchas veces su uso depende de lo que cada uno entiende por calidad, por lo cual es importante comenzar a unificar su definición.



¿QUÉ ES LA CALIDAD?

- » Las principales normas internacionales definen la calidad como :
- » “El grado en el que un conjunto de características inherentes cumple con los requisitos “ (ISO 9000)
- » “Conjunto de propiedades o características de un producto o servicio que le confieren aptitud para satisfacer unas necesidades expresadas o implícitas” (ISO 8402)



HISTORIA DE LA CALIDAD

- »La revolución industrial abrió camino en la producción, distribución de productos y de servicios.
- »En unos tiempos de crecimiento era necesario la comprobación de las piezas o servicios, es decir, el control de los procesos, antes y durante la fabricación, y a raíz de esto fueron apareciendo nuevas y varias funciones de Control De Calidad.
- »Se indica que el Control De Calidad tuvo el inicio en 1916, cuando las empresas Western Electric y la Bell Telephone unieron esfuerzos e investigaciones para la fabricación de teléfonos que pudiesen resistir con gran fiabilidad el duro uso del público.
- »En 1946 se creó la American society for Quality, una central de divulgación de la información en los temas de control de la Calidad.



HISTORIA DE LA CALIDAD

- » En 1950, en Japón, con la ayuda de W. Edwards Deming adoptó una metodología dirigida hacia el usuario, fabricando productos de calidad, es decir, - Haciendo bien las cosas a la primera -. Este estilo de metodología se iniciaba en la alta Dirección y luego extendiéndose a todos los niveles de la empresa. → TQM
- » Toda la Concientización y el seguimiento de la calidad en el producto y los procesos ha dado lugar a un buen control de calidad, planificación de la inspección, controles estadísticos, análisis de reclamaciones del mercado, prevención de los defectos y fallos, ingeniería de Calidad, fiabilidad, análisis de costes, cero defectos, control total de la Calidad, seguimiento de las ventas, aseguramiento de la Calidad, auditorias y sistemas de información de la Calidad.



CALIDAD DE LOS SISTEMAS DE INFORMACIÓN

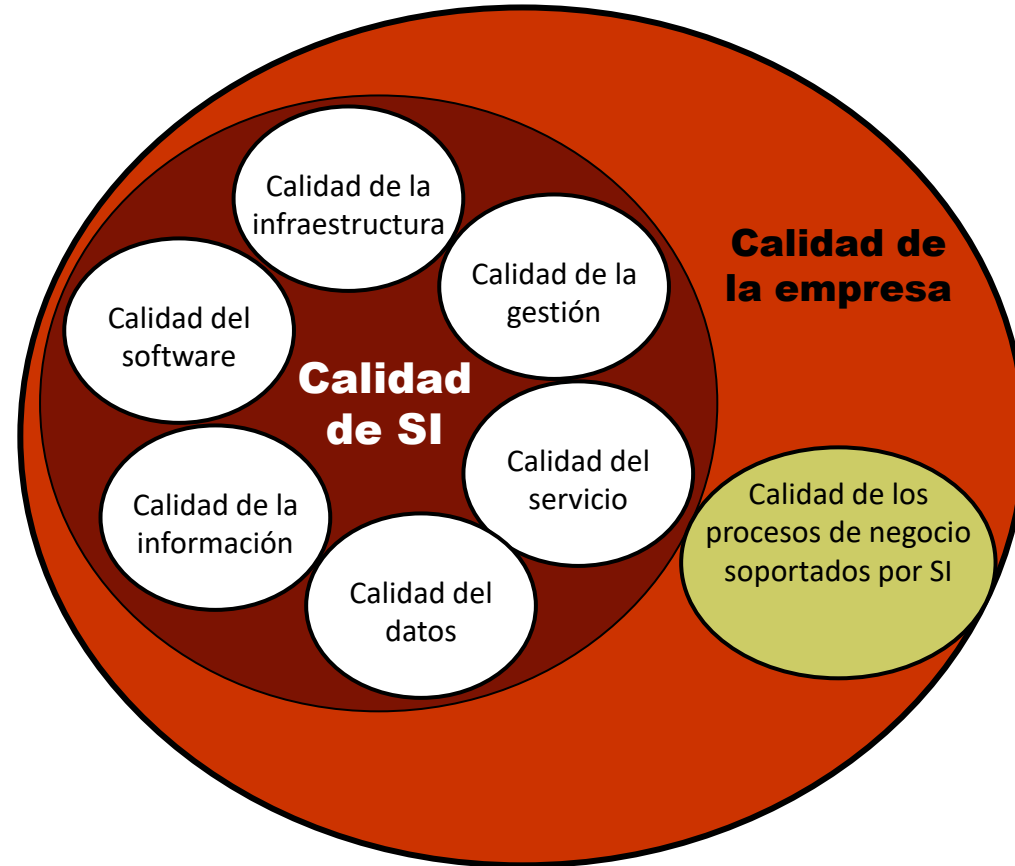
- » La importancia de los sistemas de información (SI) en la actualidad hace necesario que las empresas de tecnología hagan mucho hincapié en los estándares de calidad.
- » Stylianou y Kumar plantean que se debe apreciar la calidad desde un todo, donde cada parte que la componen debe tener su análisis de calidad.



COMPONENTES

Visión holística de la calidad

Stylianou y Kumar
(2000)



COMPONENTES

» Calidad de la Infraestructura

- incluye, por ejemplo, la calidad de las redes, y sistemas de software.

» Calidad de Software

- de las aplicaciones de software construidas, o mantenidas, o con el apoyo de IS.

» Calidad de Datos

- Que ingresan en el sistema de información.

» Calidad de Información

- está relacionada con la calidad de los datos.

» Calidad de gestión

- incluye el presupuestó , planificación y programación.

» Calidad de servicio

- incluye los procesos de atención al cliente

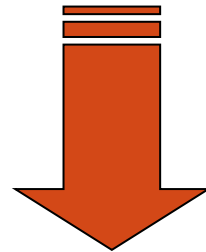


CALIDAD DE SOFTWARE

»La calidad del software se ha mejorado significativamente en estos últimos años, en particular por una mayor conciencia de la importancia de la gestión de la calidad y la adopción de técnicas de gestión de la calidad para desarrollo en la industria del software

»Se divide en

- Calidad del producto obtenido
- Calidad del proceso de desarrollo



Son dependientes

CALIDAD DEL PRODUCTO Y PROCESO

»Producto

- Un producto es un bien tangible que es el resultado de un proceso.
- Aunque el software tiene aspectos intangibles, un producto software es sin embargo un bien en sí mismo e incluye sus documentos asociados.
- La estandarización del producto define las propiedades que debe satisfacer el producto software resultante.

»Proceso

- La estandarización del proceso define la manera de desarrollar el producto software.

Sin un buen proceso de desarrollo es casi imposible obtener un buen producto.

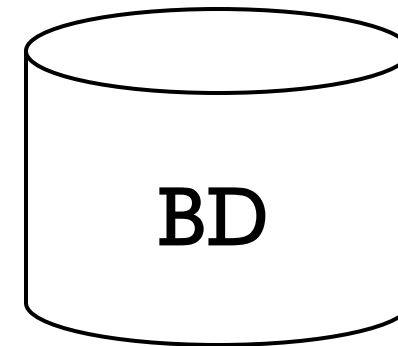
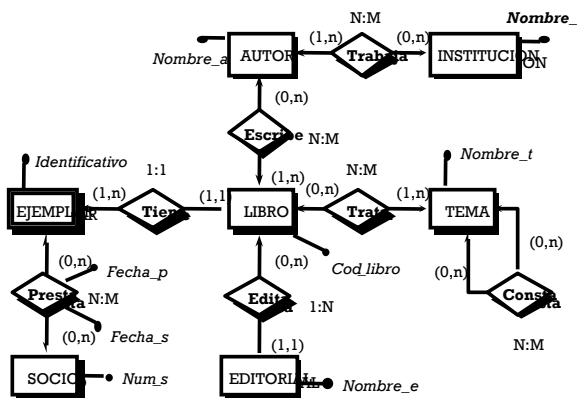


CALIDAD DEL PRODUCTO Y PROCESO

*Calidad
del
proceso*



*Calidad
del
producto*



CLASIFICACIÓN DE NORMAS Y MODELOS DE CALIDAD

Producto Software



Proceso de Desarrollo

Producto Software



Proceso de Desarrollo



Organización



CLASIFICACIÓN DE NORMAS Y MODELOS DE CALIDAD



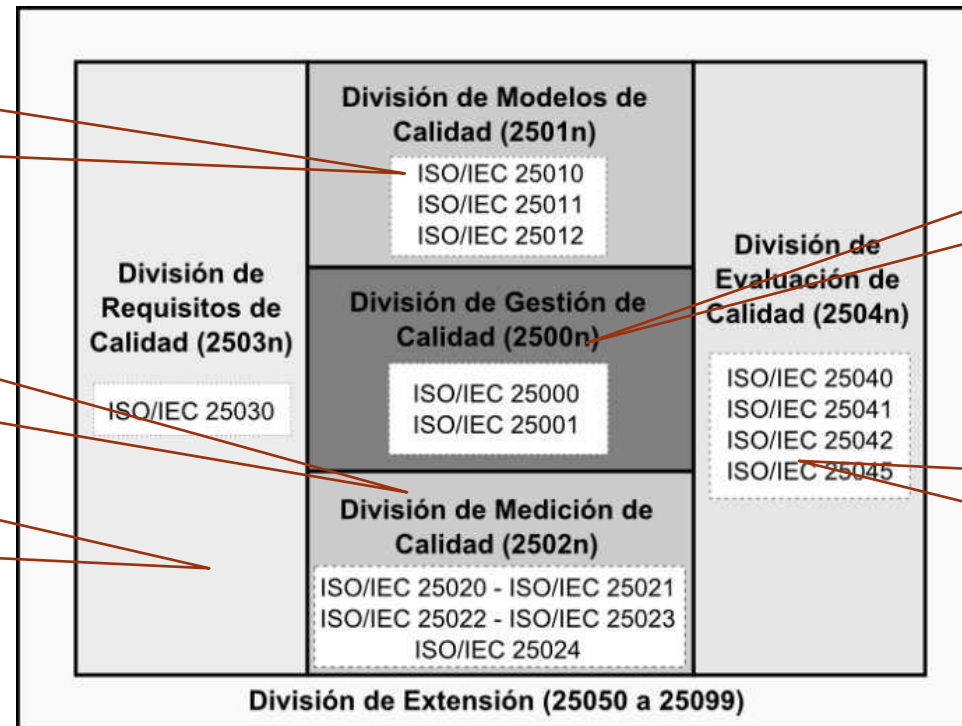
MODELO DE CALIDAD SQUARE ISO/IEC 25000

»ISO/IEC 25000 SQuaRE Software product Quality Requirement and Evaluation

Modelo de calidad detallado incluyendo características para calidad interna y externa y la calidad de datos.

Modelo de referencia de la medición de la calidad del producto, definiciones de medidas de calidad y guías prácticas de uso

Ayuda a especificar los requisitos de calidad que pueden ser usados en el proceso de elicitación.



Las normas que forman este apartado definen todos los modelos, términos y definiciones comunes referenciados por toda la serie SQuaRE

Requisitos, recomendaciones y guías para la evaluación de producto.



MODELO DE CALIDAD SQUARE ISO/IEC 25000

» ISO/IEC 2500n – División gestión de la calidad

ISO/IEC 25000:2005 - Guide to SQuaRE:

ISO/IEC 25001:2007 - Planning and Management.

» ISO/IEC 2501n – División modelos de calidad

ISO/IEC 25010 - System and software quality models

ISO/IEC 25012 - Data Quality model

» ISO/IEC 2502n – División de medición de calidad

ISO/IEC 25020 - Measurement reference model and guide

ISO/IEC 25021 - Quality measure elements

ISO/IEC 25022 - Measurement of quality in use

ISO/IEC 25023 - Measurement of system and software product quality.

ISO/IEC 25024 - Measurement of data quality

» ISO/IEC 2503n – División Requerimientos de calidad

ISO/IEC 25030 - Quality requirements

» ISO/IEC 2504n – División Evaluación de la calidad

ISO/IEC 25040 - Evaluation reference model and guide

ISO/IEC 25041 - Evaluation guide for developers, acquirers and independent evaluators

ISO/IEC 25042 - Evaluation modules.

ISO/IEC 25045 - Evaluation module for recoverability



MODELO DE CALIDAD SQUARE ISO/IEC 25010



MODELO DE CALIDAD SQUARE ISO/IEC 25010



CALIDAD DE PRODUCTO - FUNCIONALIDAD

»Ejemplo de métricas de Interoperabilidad

Table 8.1.3 Interoperability metrics

External interoperability metrics									
Metric name	Purpose of the metrics	Method of application	Measurement, formula and data element computations	Interpretation of measured value	Metric scale type	Measure type	Input to measurement	ISO/IEC 12207 SLCP Reference	Target audience
Data exchangeability (Data format based)	How correctly have the exchange interface functions for specified data transfer been implemented?	Test each downstream interface function output record format of the system according to the data fields specifications. Count the number of data formats that are approved to be exchanged with other software or system during testing on data exchanges in comparing with the total number.	X= A / B A= Number of data formats which are approved to be exchanged successfully with other software or system during testing on data exchanges B= Total number of data formats to be exchanged	$0 \leq X \leq 1$ The closer to 1.0 is the better.	Absolute	A= Count B= Count X= Count/Count	Req. spec. (User manual) Test report	6.5 Validation	Developer
	FOOTNOTE <i>It is recommended to test specified data transaction.</i>								
Data exchangeability (User's success attempt based)	How often does the end user fail to exchange data between target software and other software?	Count the number of cases a) X= 1 - A / B A= Number of cases in which user failed to exchange data with other software or systems B= Number of cases in which user attempted to exchange data		$0 \leq X \leq 1$ The closer to 1.0 is the better.	a) Absolute	A= Count B= Count X= Count/Count	Req. spec. (User manual) Test report	5.4 Operation	Maintainer
	How often are the data transfers between target software and other software successful?	b) Y= A / T T= Period of operation time		$0 \leq Y$ The closer to 0, is the better.	b) Ratio	Y= Count/Time T= Time			
	Can user usually succeed in exchanging data?								

ISO/IEC TR 9126-2:2003(E)



CALIDAD DE LOS DATOS ISO/IEC 25012

» La norma entiende por calidad de datos:

- La capacidad de las características de los datos de satisfacer necesidades explícitas e implícitas bajo determinadas condiciones de uso.

» Los clasifica estas características de calidad considerando dos puntos de vista:

- Inherente
 - Capacidad de las características de los datos de tener el potencial intrínseco para satisfacer las necesidades explícitas e implícitas
 - Este punto de vista está más relacionado con los aspectos del dominio gestionados por los expertos del negocio.
- Dependiente del sistema:
 - Capacidad del sistema informático de alcanzar y preservar la calidad de los datos cuando los datos se utilizan en determinadas condiciones
 - Este punto de vista suele ser responsabilidad de los técnicos del sistema.

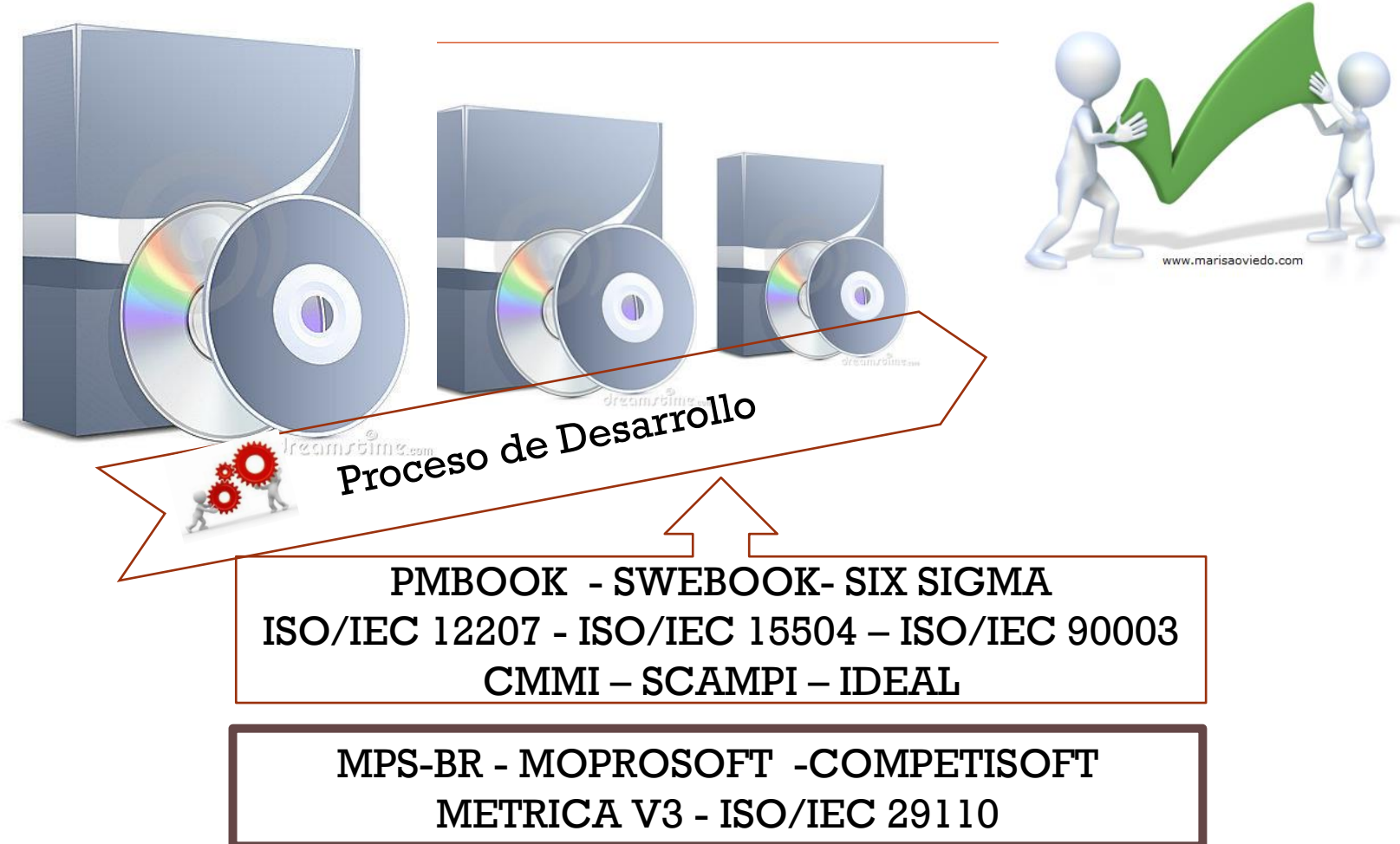


SQUARE - PROCESO DE EVALUACIÓN - ISO/IEC 25040

1. Establecer los requisitos de la evaluación
 1. Establecer el propósito de la evaluación
 2. Obtener los requisitos de calidad del producto
 3. Identificar las partes del producto que se deben evaluar
 4. Definir el rigor de la evaluación
2. Especificar la evaluación
 1. Seleccionar los módulos de evaluación
 2. Definir los criterios de decisión para las métricas
 3. Definir los criterios de decisión de la evaluación
3. Diseñar la evaluación
 1. Planificar las actividades de la evaluación
4. Ejecutar la evaluación
 1. Realizar las mediciones
 2. Aplicar los criterios de decisión para las métricas
 3. Aplicar los criterios de decisión de la evaluación
5. Finalizar la evaluación
 1. Revisar los resultados de la evaluación
 2. Crear el informe de evaluación
 3. Revisar la calidad de la evaluación y obtener feedback
 4. Tratar los datos de la evaluación



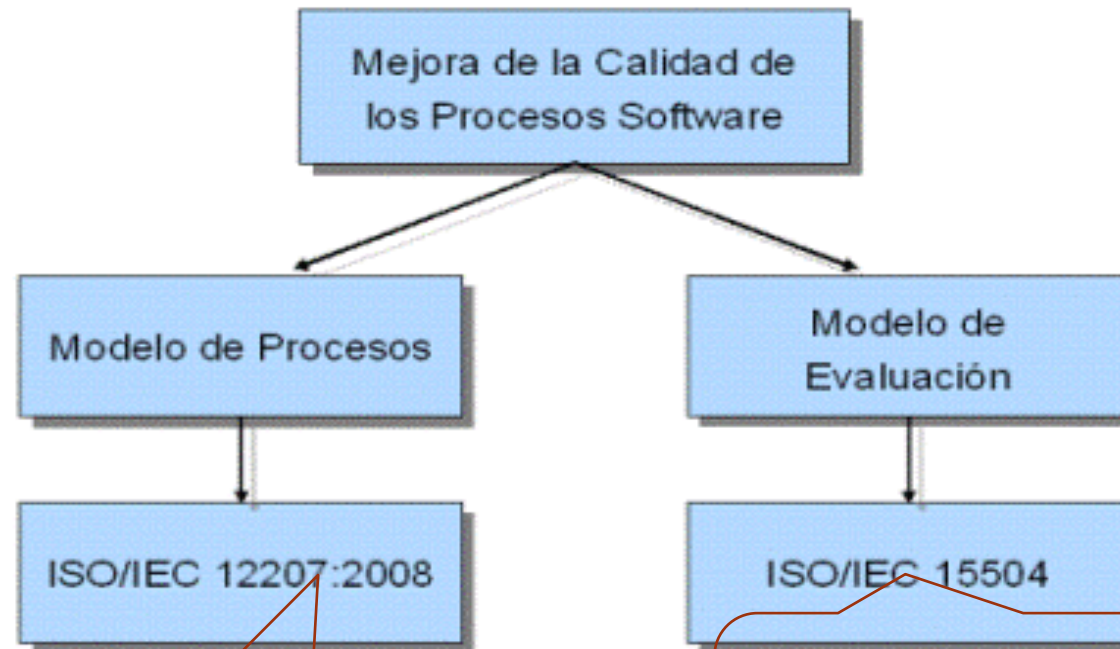
CLASIFICACIÓN DE NORMAS Y MODELOS DE CALIDAD



MODELO DE CALIDAD DE PROCESO SOFTWARE

ISO 12207/ ISO15504

»Modelo de Calidad de Proceso Software



ISO/IEC 12207 establece un modelo de procesos para el ciclo de vida del software

ISO/IEC 15504 es una norma internacional para establecer y mejorar la capacidad y madurez de los procesos de las organizaciones en la adquisición, desarrollo, evolución y soporte de productos y servicios

MODELO DE CALIDAD DE PROCESO SOFTWARE

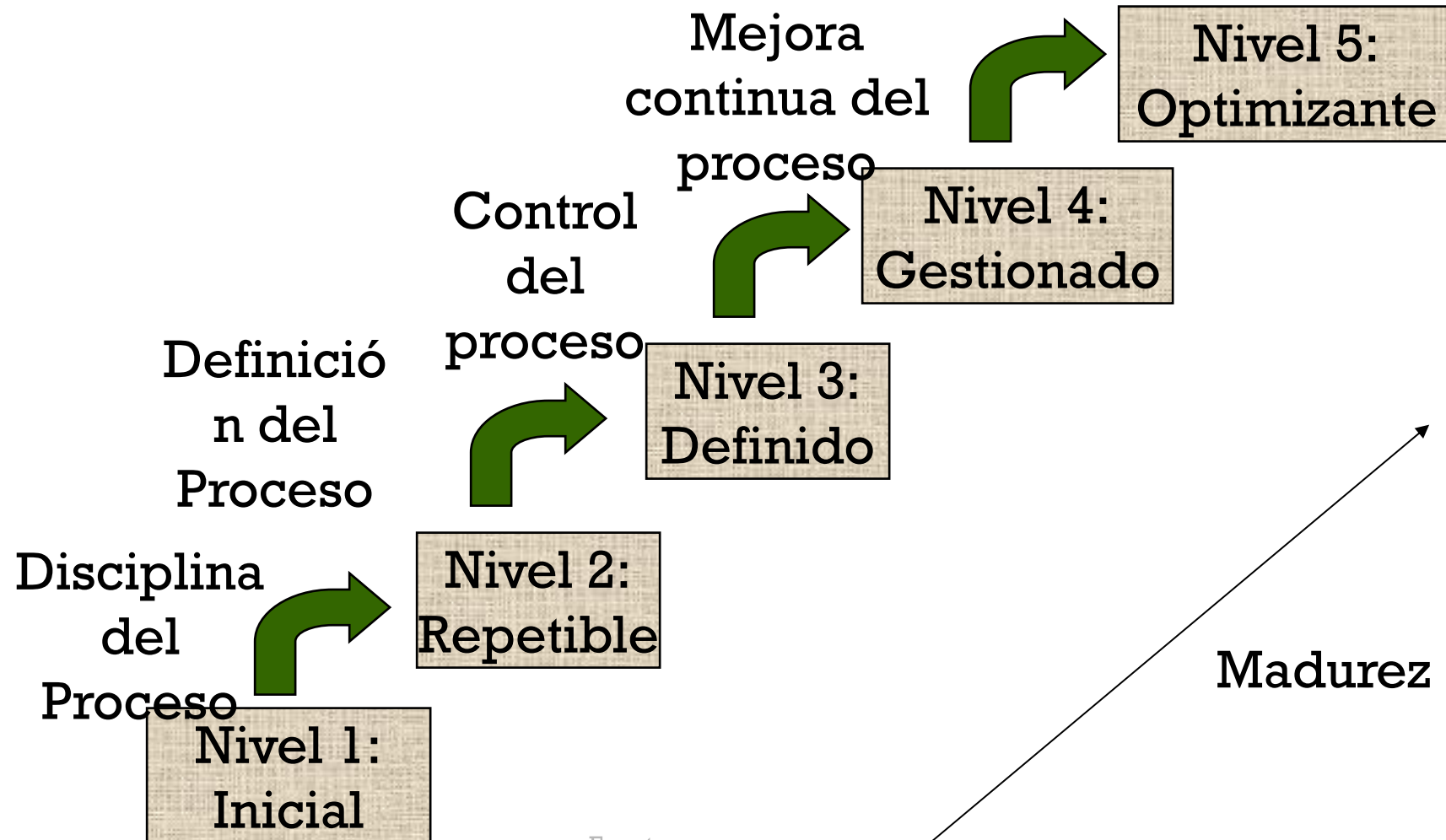
CMM - CMMI

- » En diciembre de 2000, el SEI publicó un nuevo modelo, el CMMI o "Modelo de Capacidad y Madurez - Integración", con el objetivo de realizar algunas mejoras respecto al SW-CMM (e integrarlo con el SE-CMM y el IPD-CMM, que pasaron a ser considerados como "obsoletos").
- » Incluye cuatro disciplinas, Software, Ingeniería de sistemas , Desarrollo integrado de procesos y productos y Gestión de proveedores .
- » A su vez incorpora una nueva representación, "Continua", la que permite evaluar el nivel en cada área independientemente.
- » El SEI ha desarrollado también un nuevo método de evaluación de las organizaciones según CMMI denominado SCAMPI.



CMMI

MODELO DE MADUREZ



CLASIFICACIÓN DE NORMAS Y MODELOS DE CALIDAD

Producto Software

Producto Software



Proceso de Desarrollo



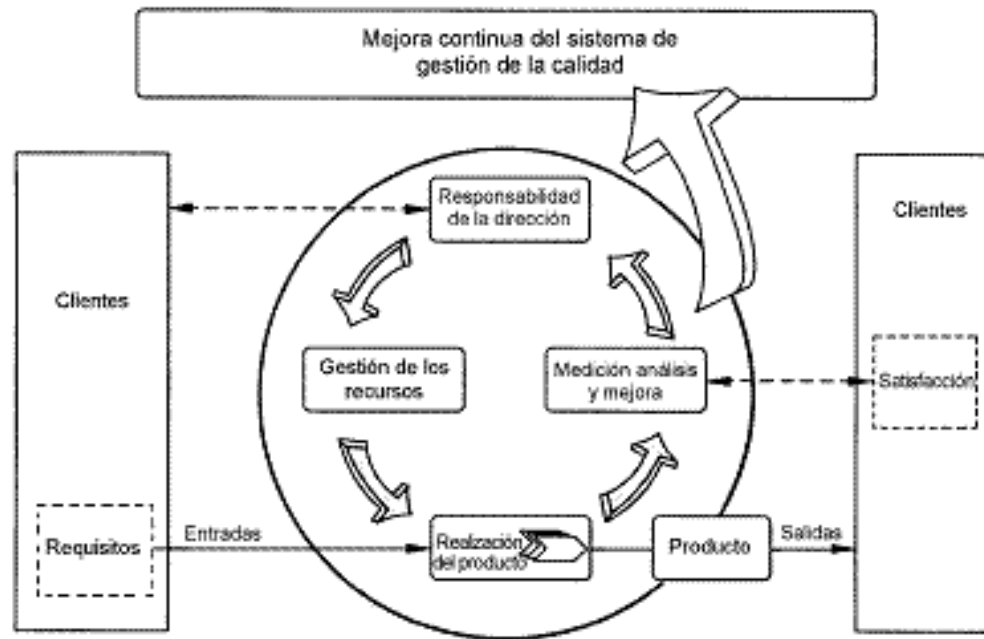
Proceso de Desarrollo

Organización

CALIDAD TOTAL – TQM – ISO/IEC 9001
SEGURIDAD DE LA INFORMACIÓN – ISO/IEC 27001

SGC – IRAM – ISO/IEC 9001

» Sistema de Gestión de la Calidad – ISO/IEC 9001



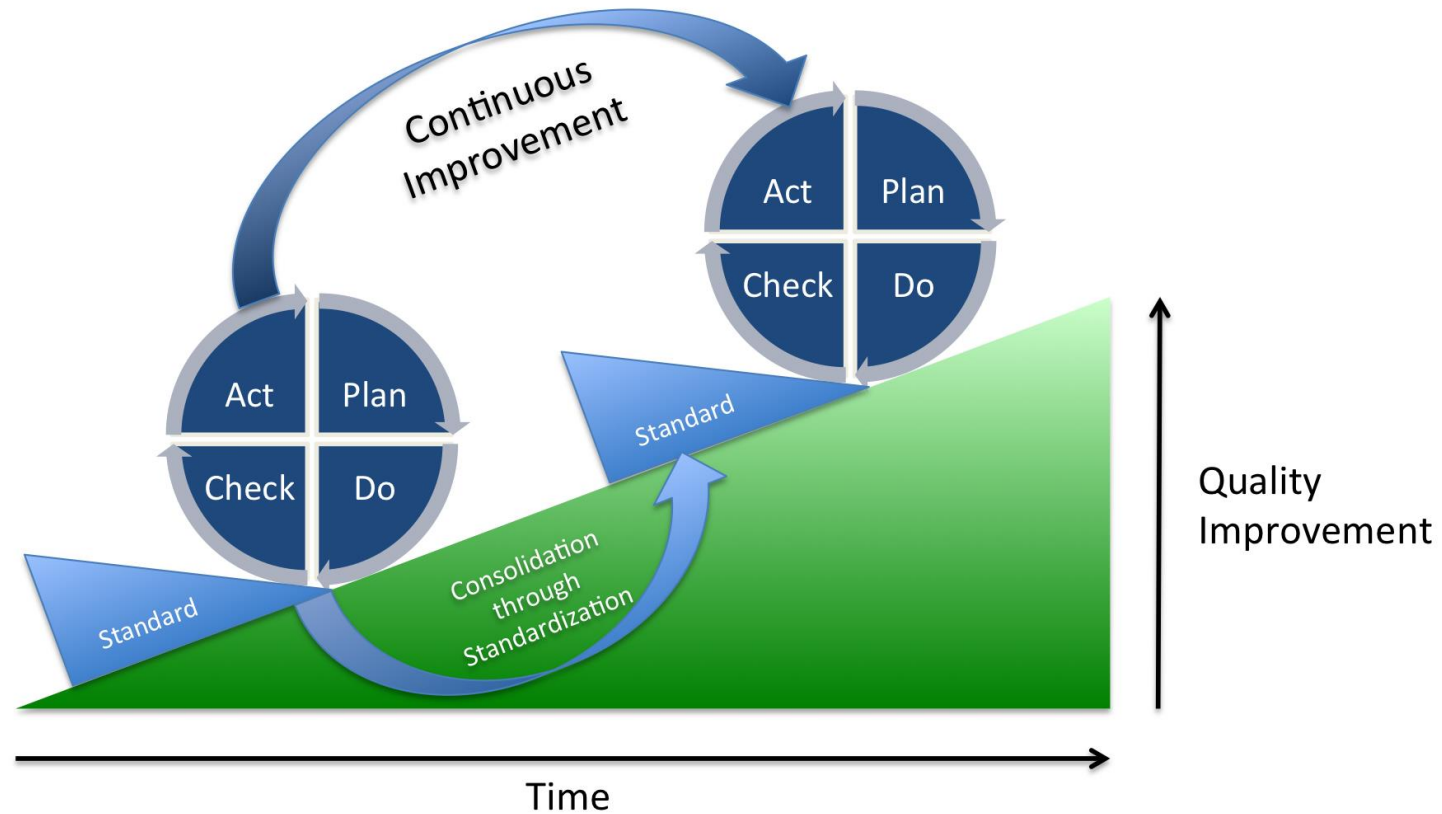
Entradas - Requisitos

Proceso

Salidas

SGC – IRAM – ISO 9001

»SGC – Mejora Continua



ISO 9000 Y EL DESARROLLO DE SOFTWARE

»IRAM – ISO 9001:2015

- Aplicación genérica

»ISO 90003:2018

- Directrices para la interpretación en el proceso de software
 - Proporciona una guía para identificar la evidencias dentro del proceso de software para satisfacer los requisitos de la ISO 9001



CLASIFICACIÓN DE NORMAS Y MODELOS DE CALIDAD

CALIDAD DE PRODUCTO DE SOFTWARE
CALIDAD DE USO – CALIDAD DE DATOS
- ISO/IEC 25000

CALIDAD DE SERVICIOS
ISO/IEC 20000 - ITIL



Orientadas a PyMEs

PMBOOK - SWEBOOK- SIX SIGMA - ISO/IEC 12207 - ISO/IEC 15504 – ISO/IEC 90003 -CMMI – SCAMPI – IDEAL - ISO/IEC 29110



CALIDAD TOTAL – TQM – ISO/IEC 9001
SEGURIDAD DE LA INFORMACIÓN – ISO/IEC 27001

